

Data Mining and Big Data Analysis

Chpter15: Recommender Systems

推荐系统

丁玥

Spring 2025

推荐系统

为什么需要推荐系统？

—如果想买一瓶饮料，有多少种办法？

- **附近24小时便利店**
  - 比较几个牌子的口碑和价格，很快就确定
- **附近的大型超市**
  - 按分类指示牌走到食品区，再按指示走到饮料区，从货架中的一堆饮料中选择自己喜欢的
- **打开饿了么、美团或者淘宝**
  - 通过搜索饮料关键字进行选择

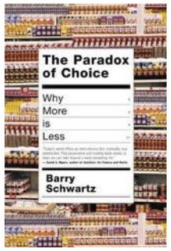
当你面对的信息越多越繁杂，那么你选择真正想要的东西就变得越困难

推荐系统

- 信息过载(information overload)
  - 作为信息消费者，如何从大量信息中找到自己感兴趣的信息是非常困难的
  - 作为信息生产者，如何让自己生产的信息脱颖而出，受到广大用户的关注，也是非常困难的

Information overload





- 互联网时代的信息爆炸
  - 10K+ movies in Netflix
  - 12M products in Amazon
  - 70M+ music tracks in Spotify
  - 10B+ videos on YouTube
  - 200B+ pins (images) in Pinterest

推荐系统

- **信息茧房（Information Cocoon）**
  - —哈佛大学教授桑斯坦在他的《信息乌托邦》里详细的阐述了这个概念
  - —指人们关注的信息领域会习惯性地被自己的兴趣所引导，从而将自己的生活桎梏于像蚕茧一般的“茧房”中的现象

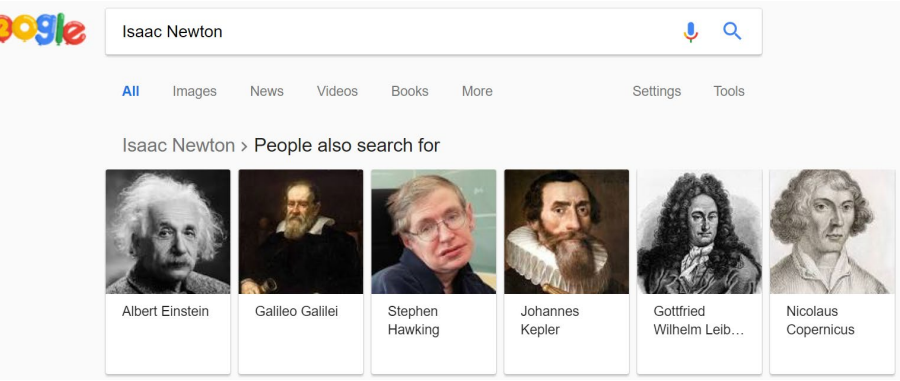


推荐系统

- 推荐系统就是解决这一矛盾的重要工具
  - —用于预测用户对物品的“评分”或“偏好”
  - —基于历史数据

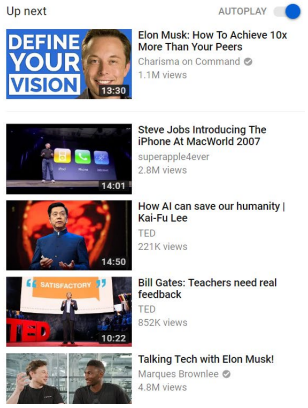


- 搜索 vs. 推荐



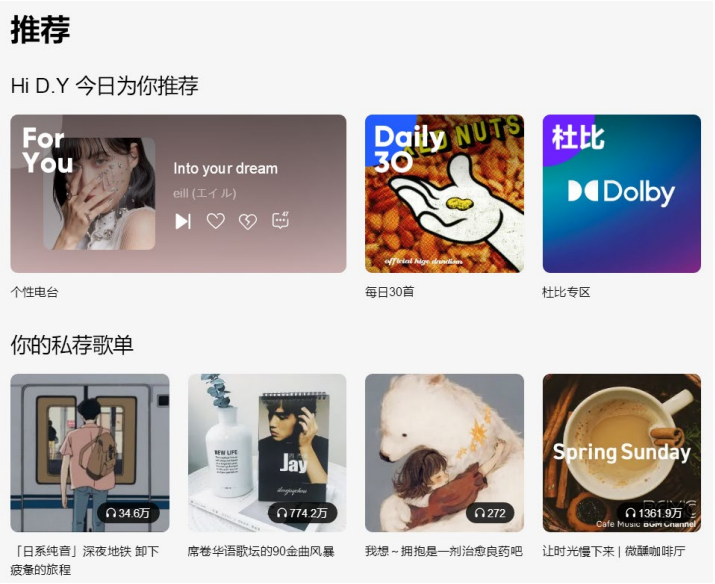


The future we're building -- and boring | Elon Musk



推荐系统

推荐系统其实无处不在



音乐推荐



电影推荐



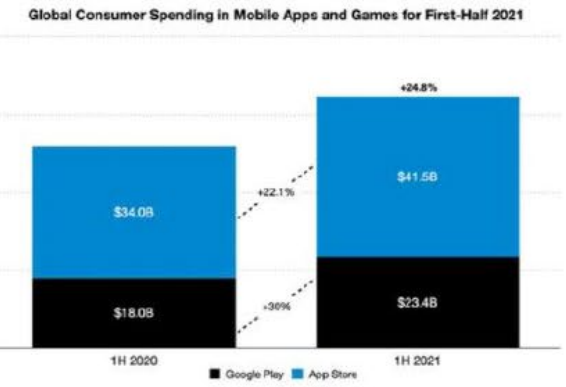
商品推荐

推荐系统

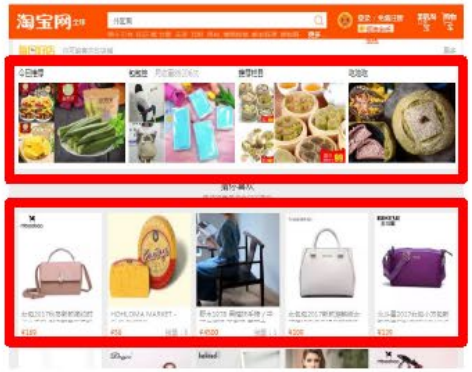

淘宝
Taobao.com
宝贝
pico
搜索
搜同款
爆款立减8.5折起 购物先领券 6.1元抢童心礼物 端午节伴手礼 整体淋浴房 毛毡板 公务一脚蹬 帕拉丁 pluggerz 香林草堂
天猫 淘宝直播 88VIP 司法拍卖 天猫超市 闲鱼集市 天猫国际 国家补贴
分类 618
抢省钱补贴 / 爆款8.5折起
电脑 / 配件 / 办公 / 文具
工业品 / 商业 / 农业 / 定制
家电 / 手机 / 通信 / 数码
家具 / 家装 / 家居 / 厨具
女装 / 男装 / 内衣 / 配饰
女鞋 / 男鞋 / 运动 / 户外
汽车 / 珠宝 / 文玩 / 箱包
食品 / 生鲜 / 酒类 / 健康
母婴 / 童装 / 玩具 / 宠物
美妆 / 个护 / 娱乐 / 图书
618狂欢节
童年拾光机
6.1元抢童心礼物
儿童节快乐
百亿补贴 · 买贵必赔 新人
¥76.9 已补30元
¥16.9 已补29.1元
¥162 已补190元
¥5.9 已补3元
淘江湖 人气热帖
《淘江湖公益百店访谈...
六一儿童节与端午节：e...
DeepSeek聊天室R1满...
《淘江湖公益访谈录》...
全网寻找'童年零食考古...
淘宝直播 红包雨
水哥珠宝
国家补贴
¥502.63
官方立减15%
淘宝秒杀 超低价
¥389
新鲜感 獭祭
猜你喜欢
家装建材 3c数码 运动户外 食品生鲜 日用百货 服饰时尚 智能家电 家用电器
360°无死角剃净 让爱亲密贴近
618 限时抢
更多低价商品

推荐系统

- 推荐系统是商业化最成功的AI技术



2021H1, App Store收入415亿美元，Google Play 234亿美元；



阿里广告收入超1745亿人民币（2019年）





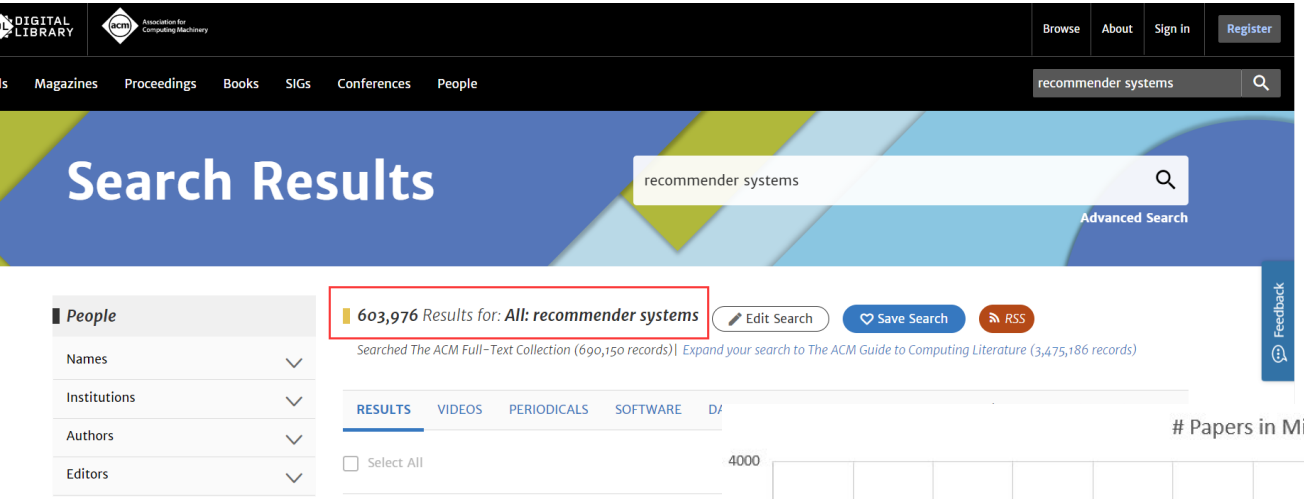
Google广告收入2297亿美元（2018年）

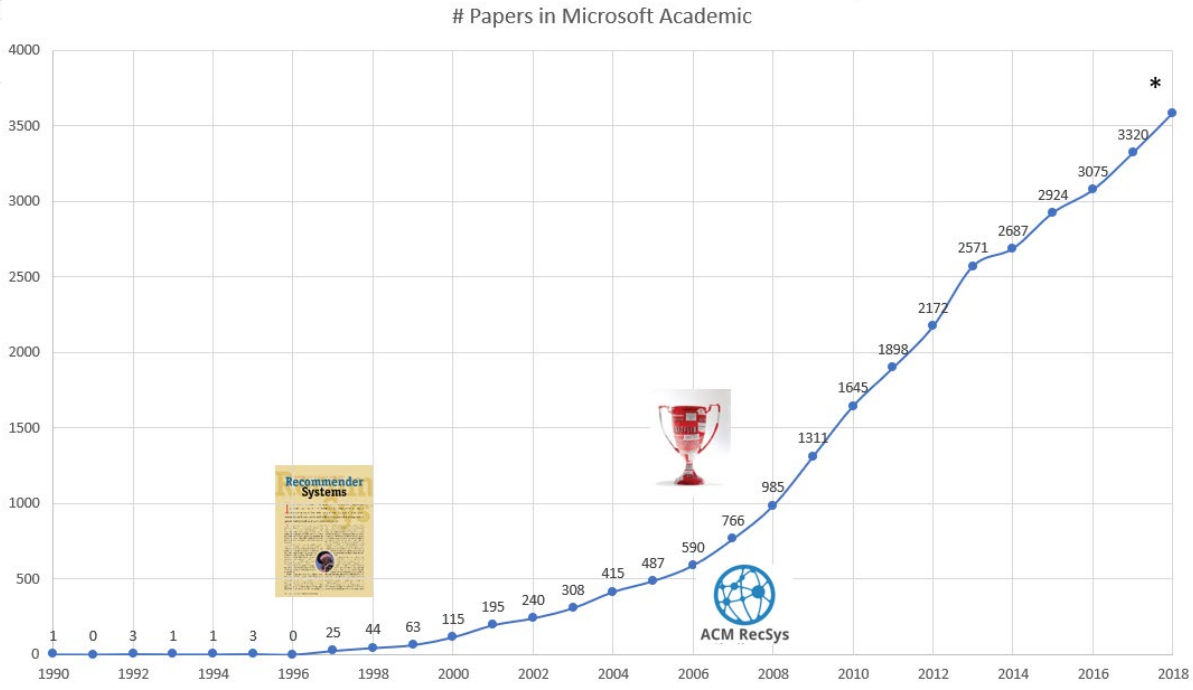
字节跳动：~1800亿人民币 (2020)

来源网上公开数据

推荐系统

- 推荐系统也受到学术界的广泛关注





推荐系统

TikTok推荐算法入选《MIT Technology Review 10》Breaking Technologies 2021

TikTok recommendation algorithms

Since its launch in China in 2016, TikTok has become one of the world's fastest-growing social networks. It's been downloaded billions of times and attracted hundreds of millions of users. Why? Because the algorithms that power TikTok's "For You" feed have changed the way people become famous online.

While other platforms are geared more toward highlighting content with mass appeal, TikTok's algorithms seem just as likely to pluck a new creator out of obscurity as they are to feature a known star. And they're particularly adept at feeding relevant content to niche communities of users who share a particular interest or identity.

The ability of new creators to get a lot of views very quickly—and the ease with which users can discover so many kinds of content—have contributed to the app's stunning growth. Other social media companies are now scrambling to reproduce these features on their own apps. **Abby Ohlheiser profiles a TikTok creator who was surprised by her own success on the platform.**

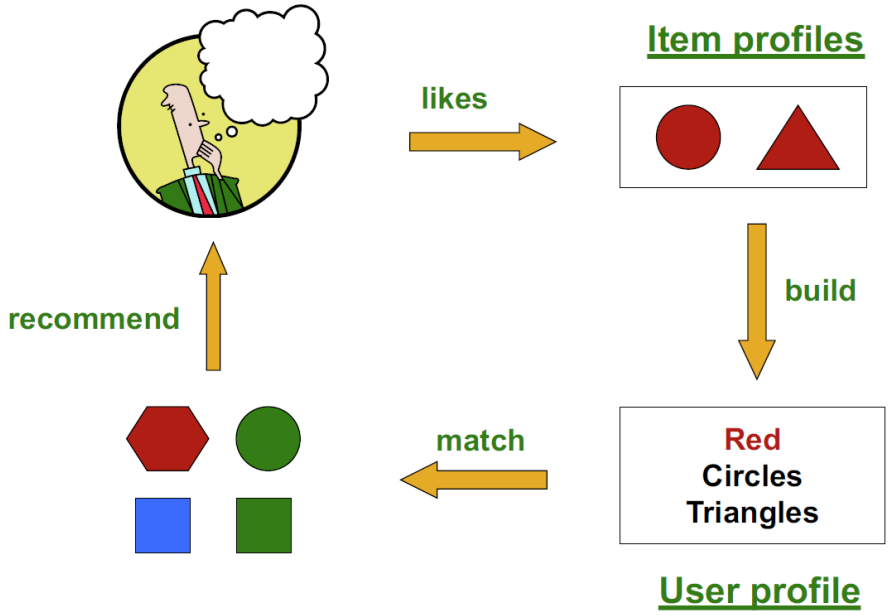
推荐系统

- 推荐系统的类型
  - —Content-based RS （基于内容）
  - —Collaborative filtering （协同过滤）

推荐系统

Content-based RS 基于内容的推荐

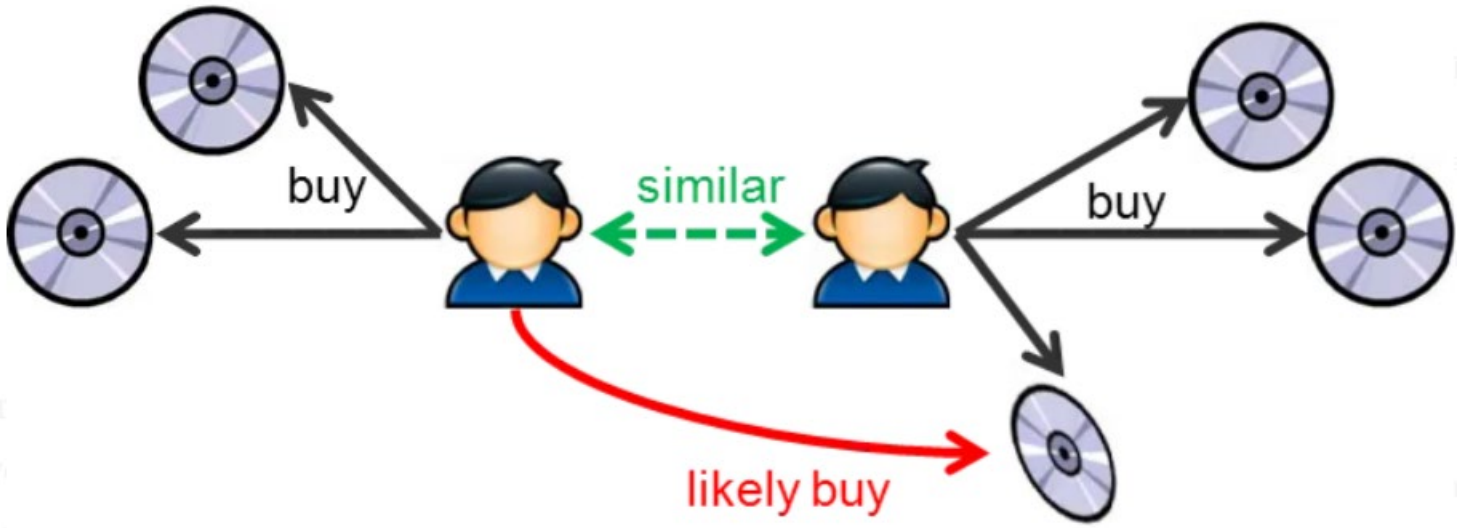
—尝试根据物品的内容（类型、颜色等）和用户的个人资料（喜好、人口统计信息等）将用户与物品进行匹配。



基于内容的过滤在向某一用户进行推荐时，不需要其他用户的数据。

推荐系统

- **Collaborative filtering 协同过滤**
  - 依赖于 “相似用户喜欢相似物品” 这一假设。推荐时会使用用户之间和 / 或物品之间的相似性度量来生成建议。



协同过滤

- 我们使用效用矩阵（也称为观测矩阵或评分矩阵）中的行和列，该矩阵包含每个用户 - 物品对的评分（或偏好）（显式或隐式）

	Flash	Arrow	Spiderman	Batman	Supergirl
User1	1	5			2
User2		5			
User3				3	
User4	1		4	3	

4×5 utility matrix for superhero ratings

- 两种协同过滤的方法:
  - —User-user CF
  - —Item-item CF

物以类聚
人以群分

基于用户的协同过滤算法

- 以基于用户的协同过滤为例
- 对于用户 x
  - *Step 1:* 找到 N个用户，他们和 x 的行为是相似的

依据什么来找“相似”

  - *Step 2:* 基于其他N个用户的行为来估计x对未交互过物品的评分

用什么方式来估计

基于用户的协同过滤算法

寻找 “相似” 用户 （搜索k-最近邻）

- 使 r_x 和 r_y 分别表示用户x 和 y 的评分
- 计算余弦相似度

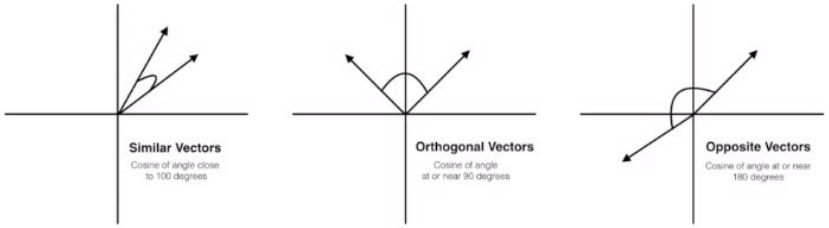
$$\text{sim}(x, y) = \cos(r_x, r_y) = \frac{r_x \cdot r_y}{||r_x|| \cdot ||r_y||}$$

r_x = [1, _, _, 1, 3]
r_y = [1, _, 2, 2, _]

r_x, r_y as points:
r_x = {1, 0, 0, 1, 3}
r_y = {1, 0, 2, 2, 0}

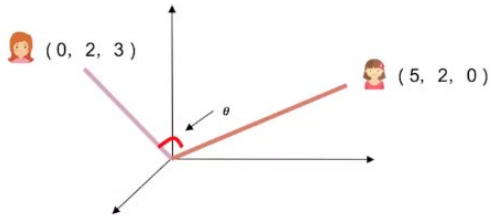
s practice!

θ	30 degrees	90 degrees	150 degrees
cosθ	0.86	0	-0.86
two vectors imilar ?	Similar	Fair / Okay / Might be not	Not Similar



Formular:

$$\cos(u_i, u_k) = \frac{\sum_{j=1}^{m} v_{ij} v_{kj}}{\sqrt{\sum_{j=1}^{m} v_{ij}^2 \sum_{j=1}^{m} v_{kj}^2}}$$



Result:

cos() = $\frac{0*5+2*2+3*0}{\text{sqrt}(0^2+2^2+3^2) * \text{sqrt}(5^2+2^2+0^2)}$ = **0.21**

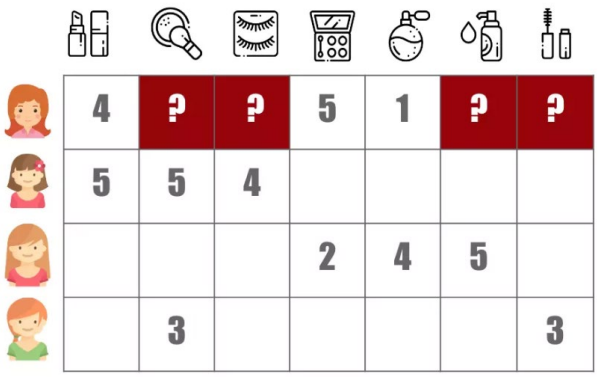
基于用户的协同过滤算法

估计评分

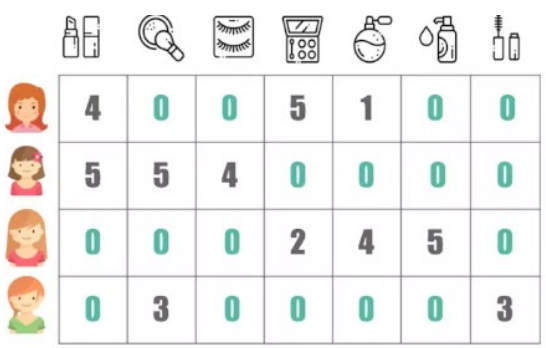
- $r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$
- 或者更进一步，加入相似度系数:
  - $r_{xi} = \frac{\sum_{y \in N} s_{xy} r_{yi}}{\sum_{y \in N} s_{xy}}$, where $s_{xy} = sim(x, y)$

基于用户的协同过滤算法

计算示例



4	?	?	5	1	?	?
5	5	4				
			2	4	5	
	3					3



4	0	0	5	1	0	0
5	5	4	0	0	0	0
0	0	0	2	4	5	0
0	3	0	0	0	0	3

P1&P2:
Cosine similarity = **0.38**
(4*5 + 0*5 + 0*4 + 5*0 + 1*0 + 0*0 + 0*0) /
[(SQRT(4^2+0^2+ 0^2+ 5^2+1^2+0^2+0^2)*
(SQRT(5^2+5^2+ 4^2+ 0^2+0^2+0^2+0^2)]

P1&P3:
Cosine similarity = **0.32**
(4*0 + 0*0 + 0*0 + 5*2 + 1*4 + 0*5 + 0*0) /
[(SQRT(4^2+0^2+ 0^2+ 5^2+1^2+0^2+0^2)*
(SQRT(0^2+0^2+ 0^2+ 2^2+4^2+5^2+0^2)]

P1&P4:
Cosine similarity = **0.0**
(4*0 + 0*3 + 0*0 + 5*0 + 1*0 + 0*0 + 0*3) /
[(SQRT(4^2+0^2+ 0^2+ 5^2+1^2+0^2+0^2)*
(SQRT(0^2+3^2+ 0^2+ 0^2+0^2+0^2+3^2)]

1 Assign average score to each person

Person 1: average score = (4+5+1) = 10/3



4	10/3	10/3	5	1	10/3	10/3
5	5	4	14/3	14/3	14/3	14/3
11/3	11/3	11/3	2	4	5	11/3
11/3	3	11/3	11/3	11/3	11/3	3

2 Normalize

Formular: Score - Average(Scores)

Eg: Person 1 score: 4 - 10/3 = 0.7



0.7	0.0	0.0	1.7	-2.3	0.0	0.0
0.3	0.3	-0.7	0.0	0.0	0.0	0.0
0.0	0.0	0.0	-1.7	0.3	1.3	0.0
0.0	0.0	0.0	0.0	0.0	0.0	0.0

推荐系统

- 协同过滤方法的优缺点
  - —计算简单
  - —可解释性好
  - —大规模计算时效率低，存在较大空间复杂度
  - —无法应对冷启动

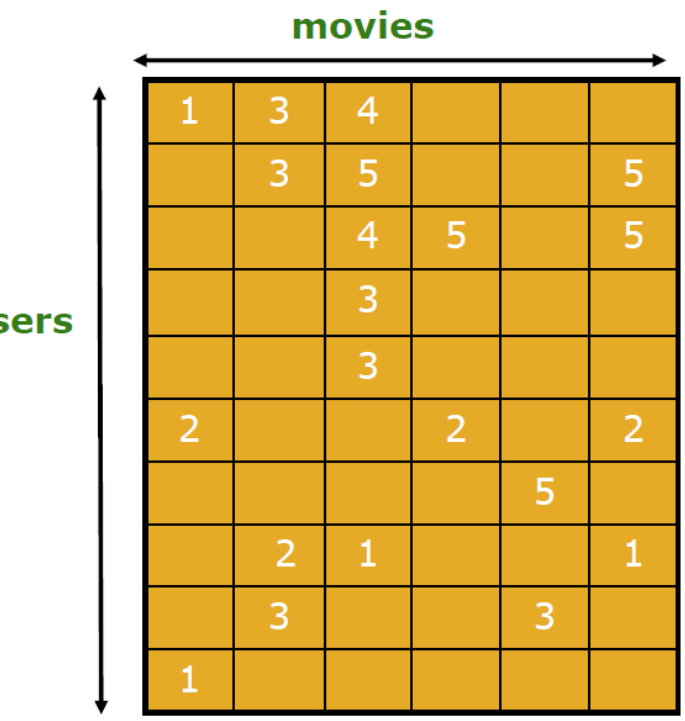
推荐系统

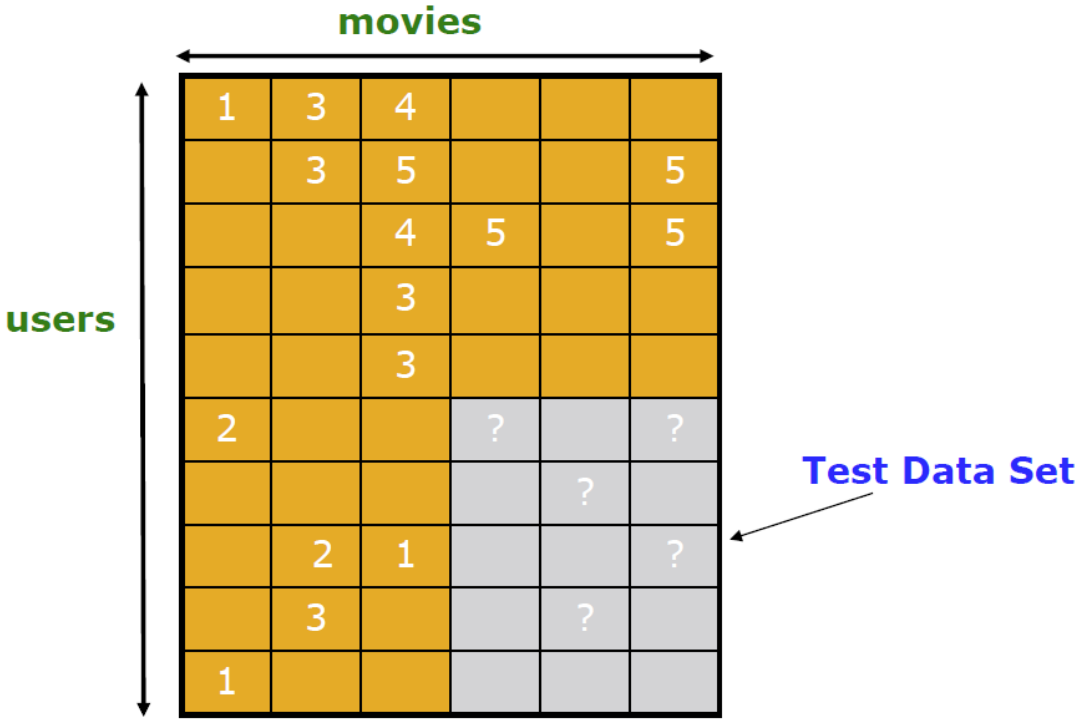
- **评测指标**
  - —用于评价推荐系统各方面的性能
  - —有些可以定量计算，有些只能定性描述，有些可以通过离线实验计算，有些需要通过用户调查获得，还有些只能在线评测

推荐系统

评测指标

—定量计算





推荐系统

评测指标

—**预测准确度。从**用户对物品的历史评分，就可以从中习得用户的兴趣模型，并预测该用户在将来看到一个他没有评过分的物品时，会给这个物品评多少分。预测用户对物品评分的行为称为评分预测。

—评分预测的预测准确度一般通过**均方根误差（RMSE）**和**平均绝对误差（MAE）**计算

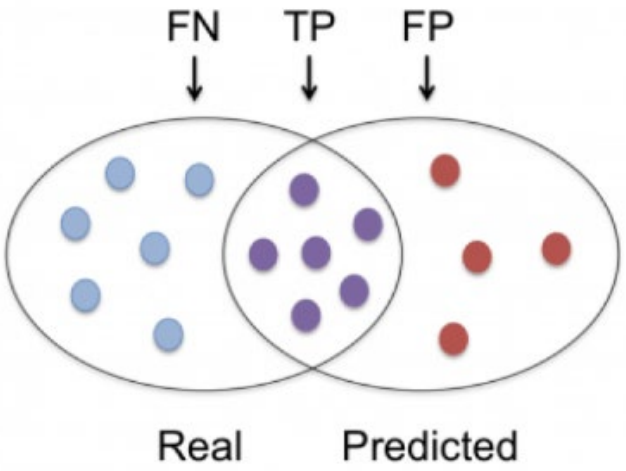
$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2}$$

$$\mathrm{MAE} = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j|$$

推荐系统

评测指标

—**TopN 推荐。**网站在提供推荐服务时，一般是给用户一个个性化的推荐列表，这种推荐叫做TopN推荐。TopN推荐的预测准确率一般通过**准确率（precision）/召回率（recall）**度量。



(a) Precision = 0.6, Recall = 0.5

$$\text{Recall@}N = \frac{|R \bigcap \hat{R}_{1:N}|}{|R|} = \frac{TP}{TP + FN}$$

$$\text{Prec@}N = \frac{|R \bigcap \hat{R}_{1:N}|}{N} = \frac{TP}{TP + FP}$$

$$\text{F1 Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{TP}{TP + \frac{1}{2}(FP + FN)}$$

推荐系统

关于评分预测和 TopN 推荐的讨论

- 评分预测是推荐系统早期研究的热点，绝大多数推荐系统的研究都是基于用户评分数据的评分预测。
  - 一方面推荐系统的早期研究组GroupLens的研究主要就是基于电影评分数据MovieLens进行的，
  - Netflix大赛也主要面向评分预测问题。因而，很多研究人员都将研究精力集中在优化评分预测的RMSE上。
- 对此，亚马逊前科学家Greg Linden有不同的看法。他指出电影推荐的目的是**找到用户最有可能感兴趣的电影，而不是预测用户看了电影后会给电影什么样的评分**。因此，TopN推荐更符合实际的应用需求。
- 也许有一部电影用户看了之后会给很高的分数，但用户看的可能性非常小（**即排序很靠后**）。因此，预测用户是否会看一部电影，应该比预测用户看了电影后会给它什么评分更加重要

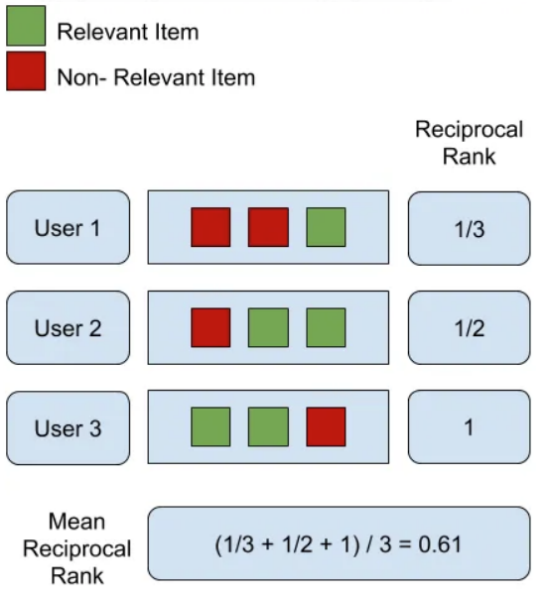
推荐系统

- 关于评分预测和 TopN 推荐的讨论
- 考虑一个case：
  - 对于两个item的评分进行预测，ground truth为i_1=4, i_2=5
  - 有两个预测模型A和B，输出的预测结果分别为2、7以及5、4
  - 从排序指标看，模型A优于模型B，从RMSE和MAE指标看，模型B缺优于模型A
  - 我们应该更加关注排序的结果，而不是预测实际用户的评分会是多少

推荐系统

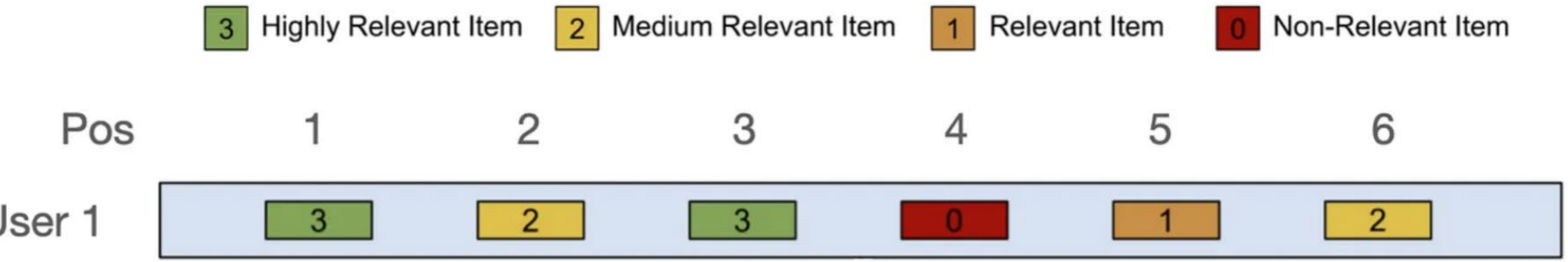
- 其他评估指标
- 平均排名倒数：(Mean reciprocal rank ， MRR)
  - 最早在搜索场景下衡量返回结果的质量，并且基于一个假设“每一个query Q”只有一个与其相关(relevant)的结果A
  - 强调所找到的item是否被放在用户更明显的位置

$$MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{p_i}$$



推荐系统

- 标准化折扣累积增益(Normalized Discounted Cumulative Gain, NDCG)
- 累积增益(Cumulative Gain, CG):



$$\text{Cumulative Gain} = 3 + 2 + 3 + 0 + 1 + 2 = 11$$

推荐系统

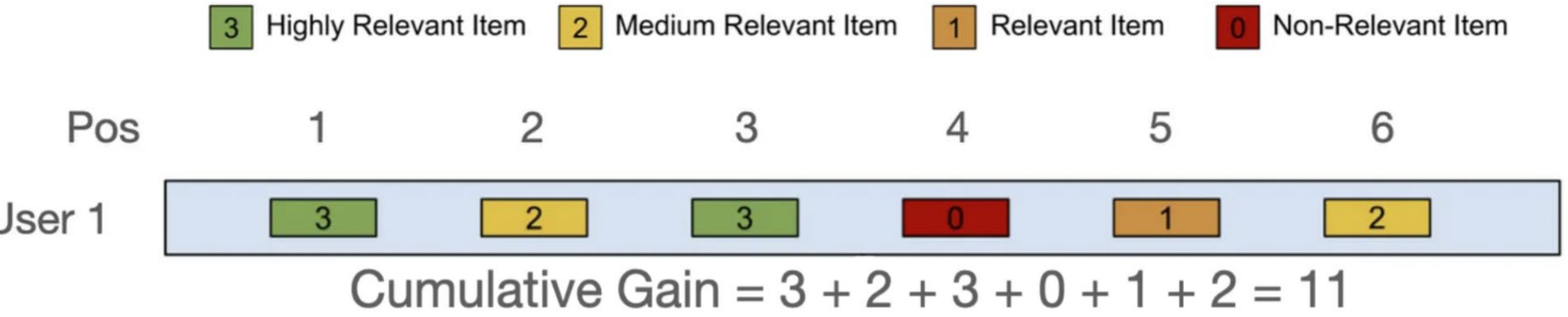
- 在这个例子中，显然位置1所放置Item的重要性大于位置3，因此对位置的权重做调整：Discounted Cumulative Gain (DCG)

标准版

$$\mathrm{DCG_p} = \sum_{i=1}^{p} \frac{rel_i}{\log_2(i+1)}$$

业界版

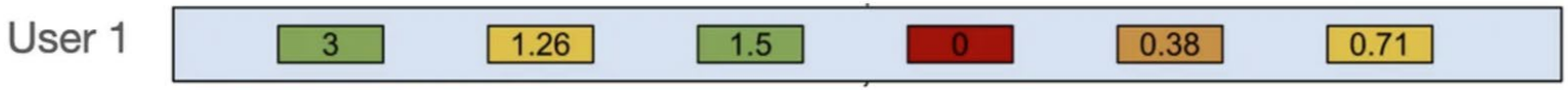
$$\mathrm{DCG_p} = \sum_{i=1}^{p} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$



推荐系统

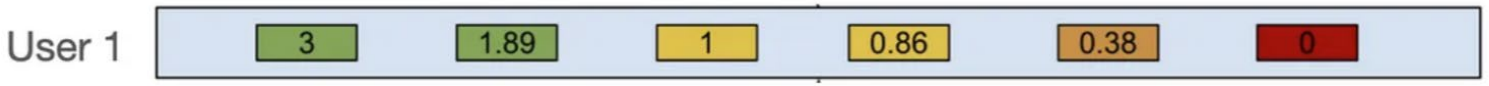
- 标准化折扣累积增益(Normalized Discounted Cumulative Gain, NDCG)
- 以标准版的公式为例，计算DCG

scounted Cumulative Gain(standard) = 3/log2 + 2/log3 + 3/log4 + 0/log5 + 1/log6 + 2/log7 = 6.8



由于用户的评分均值存在差异（跟别的用户比是不客观的），需要找到一个标准来衡量“DCG=6.8”到底是高还是低：**Ideal DCG——理想排序下的DCG值**

eal Discounted Cumulative Gain(standard) = 3/log2 + 3/log3 + 2/log4 + 2/log5 + 1/log6 + 0/log7 = 7.141



推荐系统

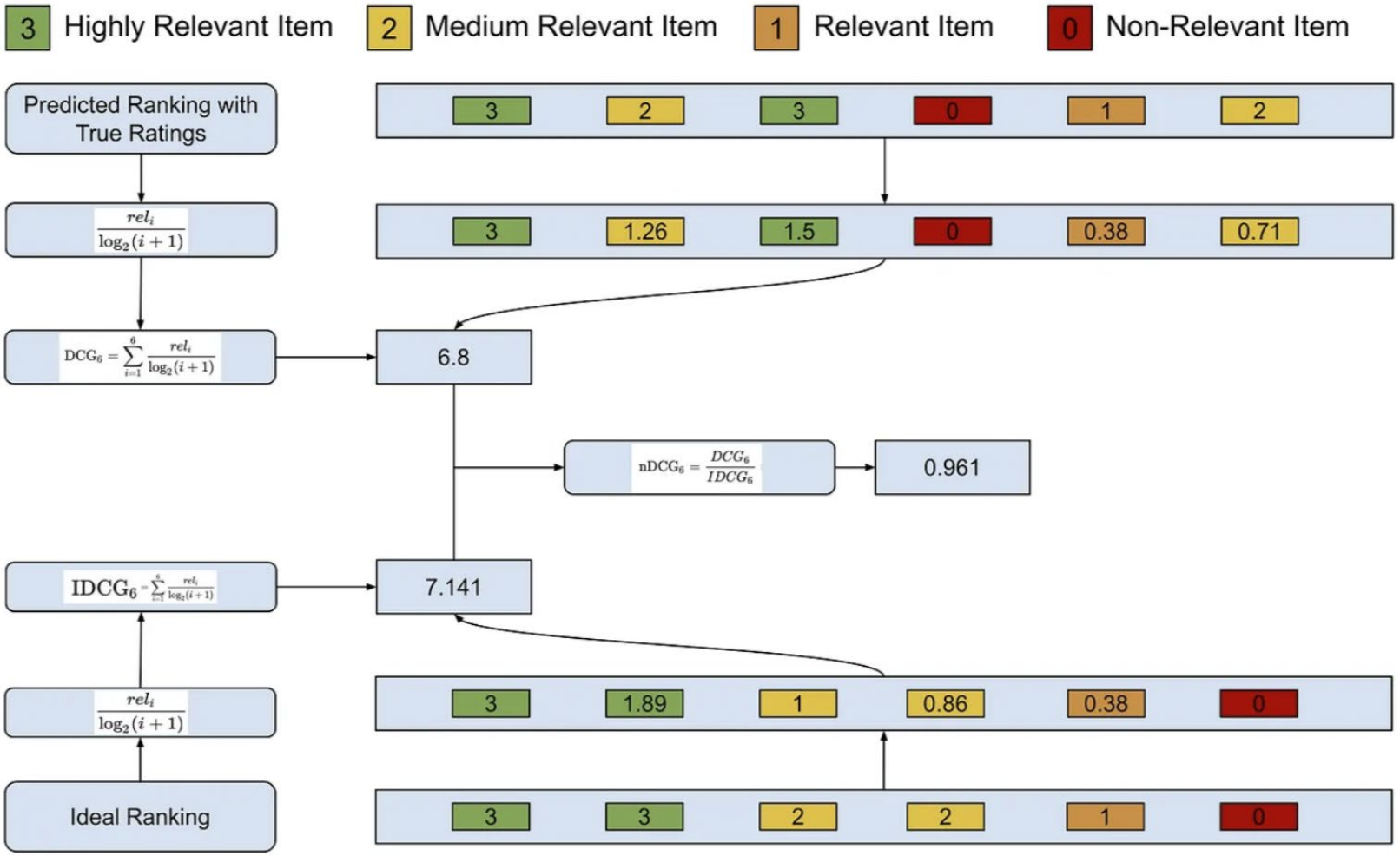
- 标准化折扣累积增益(Normalized Discounted Cumulative Gain, NDCG)
  - 我们得到Normalized DCG的值（只是单个user的）

Normalized Discounted Cumulative Gain(standard) = 6.8/7.141 = 0.961

  - 最后我们把全部user的NDCG值相加并取平均就是最终的NDCG值

推荐系统

- 标准化折扣累积增益(Normalized Discounted Cumulative Gain, NDCG)



推荐系统

实例

```
# 假设 top-20 推荐,一共 5 个 user, 50 个 item ,隐式反馈数据集.
users = np.array([0, 1, 2, 3, 4])
# test_set 表示 5 个用户在测试集中分表交互过那些 item
test_set = [
    [0, 21, 31, 41, 49],
    [2, 3, 4, 5, 33],
    [5, 10, 20, 30, 39, 44, 45, 49],
    [4, 7, 13, 15],
    [2]
]
```

U_1 Recommendation list (Top-10):
[0,4,2,7,5,8,3,1,6,9]
Recall=4/5=0.8, Precision=4/10=0.4

U_1rel = [0,1,1,0,1,0,1,0,0,0]

U_1 sorted = [2,3,4,5,0,7,8,1,6,9]

U_1 sorted rels = [1,1,1,1,0,0,0,0,0,0]

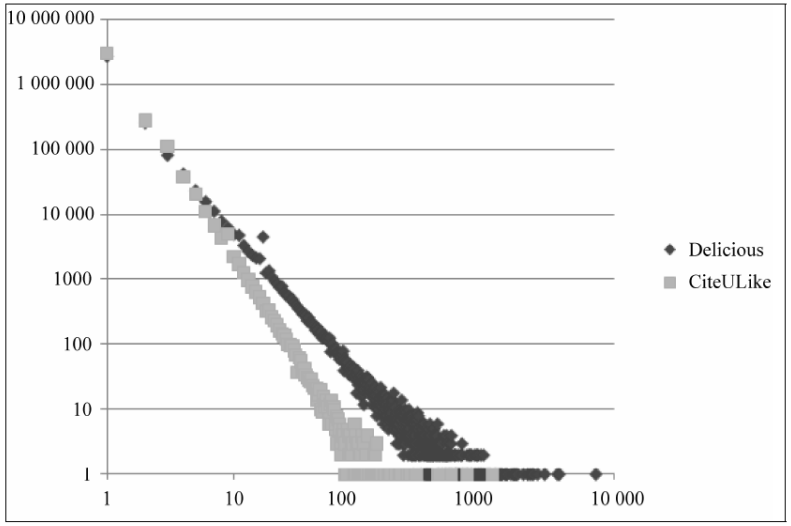
```
rels = rel_matrix[user]
dcg = DCG(rels)
idcg = DCG(sorted(rels, reverse=True))
```

```
# 获得隐式反馈的rel分数矩阵
rel_matrix = get_implict_matrix(rec_items, test_set)
```

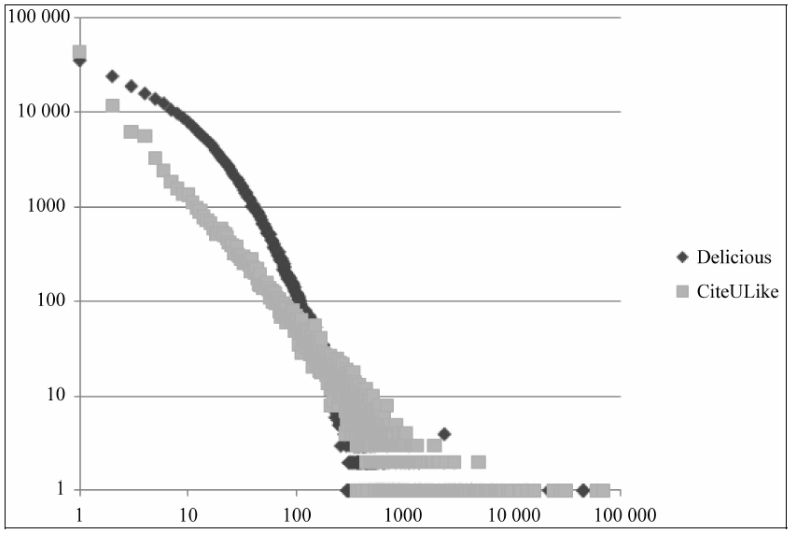
推荐系统

- **推荐系统的一些挑战**
  - **长尾(long-tail)问题**
  - **马太效应(Matthew effect)：**热门头部物(Head Items)的样本量多，模型学习这部分的效果越好，而长尾物品的样本量少，导致模型对该部分Item的理解不够充分，效果自然也就较差





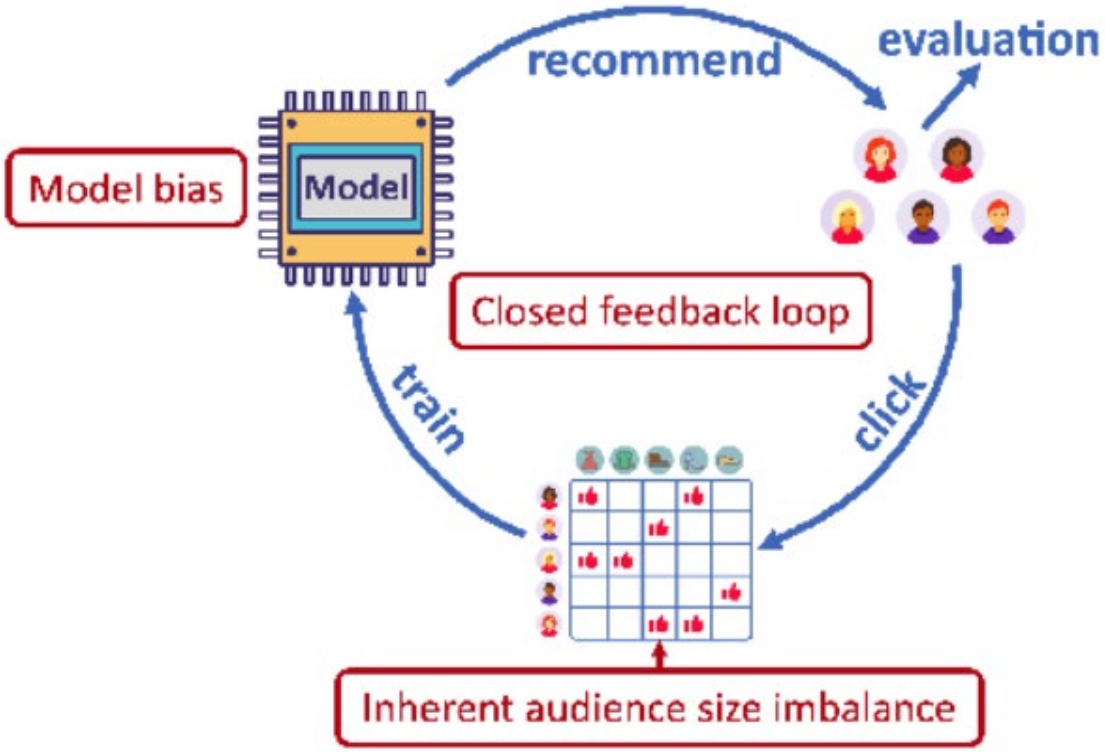
物品流行度的长尾分布



用户活跃度的长尾分布

推荐系统

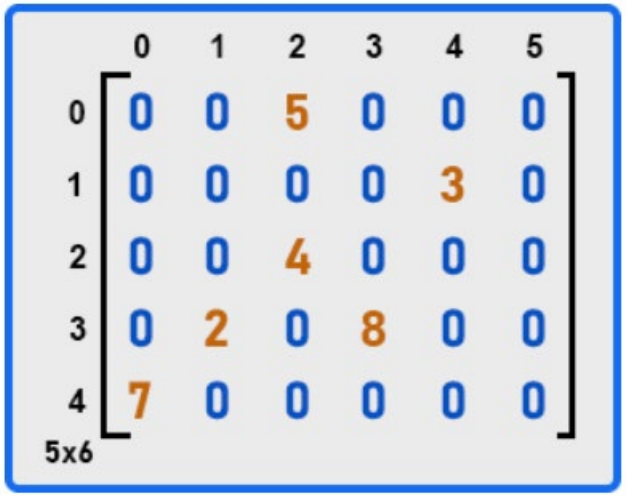
- 流行度偏差（Popularity bias）
  - —**热门商品的推荐频率甚至超过了它们的受欢迎程度，模型通常会给热门项目的评分高于其理想值**，而只是简单地将不受欢迎的商品预测为负值
  - —**这种偏差会形成闭环并放大**



推荐系统

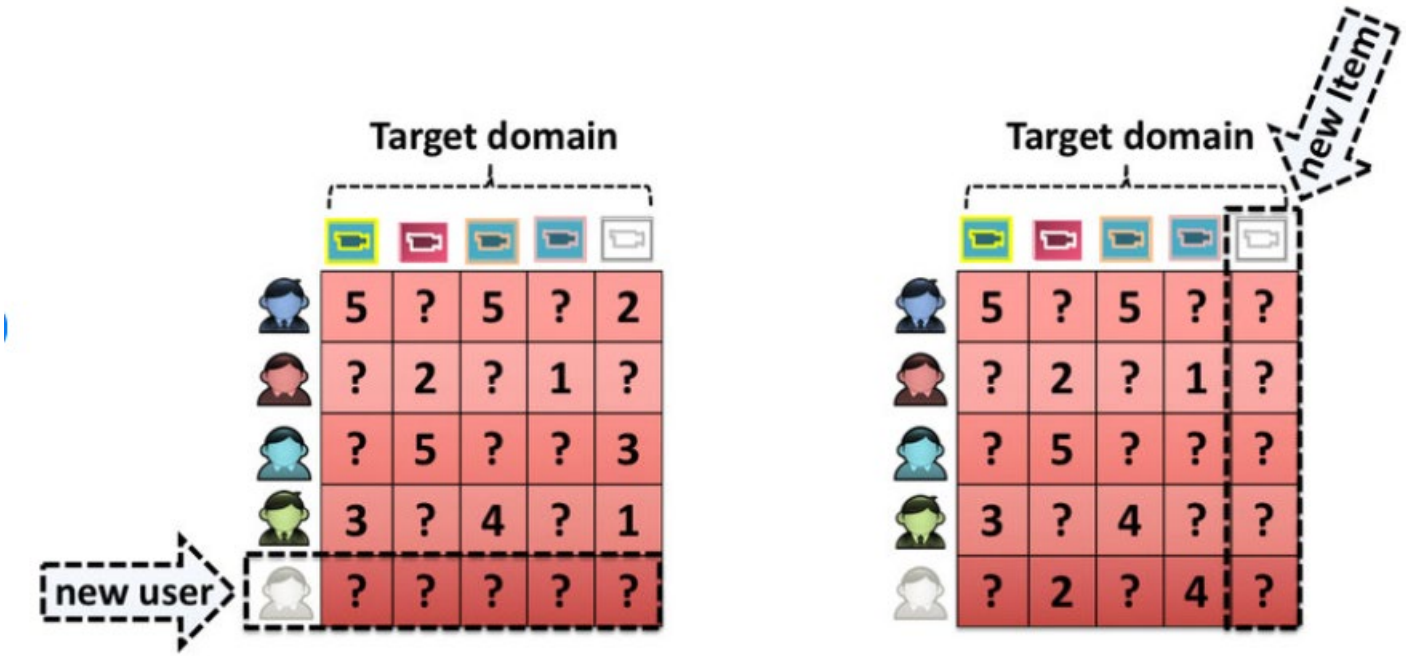
稀疏性问题(sparsity)

- Utility matrix规模大且极度稀疏
  - E.g, 10000个user，10000个item，10000个评分，sparsity = 1-1/10000=99.9999%
- 对算法提出了很大的挑战
  - 协同过滤方法失效
  - 基于机器学习的模型容易产生过拟合



推荐系统

- **冷启动问题(cold-start)**
  - 如果是新开发的产品，初期用户很少，用户行为也不多，常用的协同过滤、深度学习等依赖大量用户行为的算法不能很好地训练推荐模型

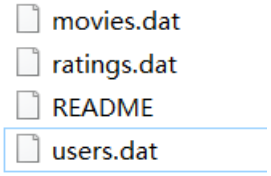


参考练习

- 数据集 Movielens 1M
  - https://grouplens.org/datasets/movielens/
  - 用户数 6040
  - 物品数 3952
  - 评分1-5
  - 交互数 1,000,209

参考练习

- **Ratings.dat**
  - —User, item, rating, timestamp
- **Movies.dat和users.dat是辅助信息，可以不用**
- **实现过程：**
  - — 数据预处理（清洗）
  - — 将ratings.dat划分为训练集和测试集，比例为8：2
    - 为了数据的平衡，按每个用户评分记录的8：2来划分
    - 保留20%测试集
    - 可以用python的pandas包来完成数据预处理，最后一列timestamp可以去掉
  - — 模型结果在测试集上进行验证




```
1::1193::5::978300760
1::661::3::978302109
1::914::3::978301968
1::3408::4::978300275
1::2355::5::978824291
1::1197::3::978302268
1::1287::5::978302039
1::2804::5::978300719
1::594::4::978302268
1::919::4::978301368
1::595::5::978824268
1::938::4::978301752
1::2398::4::978302281
1::2918::4::978302124
1::1035::5::978301753
1::2791::4::978302188
1::2687::3::978824268
1::2018::4::978301777
1::3105::5::978301713
1::2797::4::978302039
1::2321::3::978302205
1::720::3::978300760
1::1270::5::978300055
1::527::5::978824195
1::2340::3::978300103
1::48::5::978824351
1::1097::4::978301953
1::1721::4::978300055
1::1545::4::978824139
```

序号、用户、物品、评分、时间戳

```
4  622678,3769,2123,4,966637216
5  869699,5254,3263,3,961290281
6  628249,3805,593,5,965966732
7  899964,5441,1083,4,959989523
8  746169,4451,562,3,965091655
```

参考练习

- 参考练习：
  - 1. 计算用户之间相似度 (cosine similarity，或自行选择相似度函数)
  - 2. 为每个选取k个相似用户（自定义k），并根据top-k相似用户的交互记录生成当前用户的预测结果并排序
  - 3. 在测试集上评估结果 （recall, precision）

参考练习

- Github上几个可以参考的项目
  - https://github.com/ChangUk/pyCollaborativeFiltering#readme
  - https://github.com/revantkumar/Collaborative-Filtering

参考练习

- Tips
  - python3.x，根据需要安装numpy,sklearn等包
  - 处理数据可以使用pandas包
  - 注意dataloader是如何导入数据的，格式上的处理、训练集和验证集的划分等
  - 注意id溢出的问题，i.e., 用户id为0-1000，程序里面1-1001
  - 相似度的计算，设置最近邻的个数
  - 评估指标
  - 区分训练集和测试集

参考练习

- 代码示例
  - —dataloader

```
import os


def loadData(filePath, inv = False):
    '''
    Load data from a input file into memory with dictionary format.
    * Input file format: userID \t itemID \t rating \n
    * Output data format: {userID: {itemID: rating, ...}, ...}
    '''
    data = {}
    try:
        with open(filePath) as file:
            for line in file:
                line = line.replace("\n", "")
                tokens = line.split("\t")

                if len(tokens) < 2:
                    continue
                elif len(tokens) == 2:
                    user = tokens[0]
                    item = tokens[1]
                    rating = 1
                else:
                    user = tokens[0]
                    item = tokens[1]
                    rating = tokens[2]

                # Store data
                if inv == False:
                    data.setdefault(user, {})
                    data[user][item] = float(rating)
                else:
                    data.setdefault(item, {})
                    data[item][user] = float(rating)
            file.close()
```

参考练习

代码示例

—相似度计算

```
ef similarity_user(data):
        print "Hello User"
        #f_i_d = open("sim_user_based.txt","w")
        user_similarity_cosine = np.zeros((users,users))
        user_similarity_jaccard = np.zeros((users,users))
        user_similarity_pearson = np.zeros((users,users))
        for user1 in range(users):
                print user1
                for user2 in range(users):
                        if np.count_nonzero(data[user1]) and np.count_nonzero(data[user2]):
                                user_similarity_cosine[user1][user2] = 1-scipy.spatial.distance.cosine(data[user1],data[user2])
                                user_similarity_jaccard[user1][user2] = 1-scipy.spatial.distance.jaccard(data[user1],data[user2])
                                try:
                                        if not math.isnan(scipy.stats.pearsonr(data[user1],data[user2])[0]):
                                                user_similarity_pearson[user1][user2] = scipy.stats.pearsonr(data[user1],data[user2])[0]
                                        else:
                                                user_similarity_pearson[user1][user2] = 0
                                except:
                                        user_similarity_pearson[user1][user2] = 0

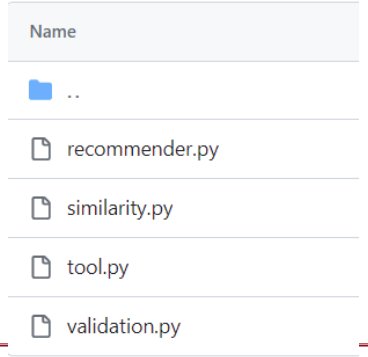
                        #f_i_d.write(str(user1) + "," + str(user2) + "," + str(user_similarity_cosine[user1][user2]) + "," + str(user_similarity_jaccard[user1][use
        #f_i_d.close()
        return user_similarity_cosine, user_similarity_jaccard, user_similarity_pearson
```

参考练习

Usage example

User-based Recommendation

```
>>> import tool
>>> data = tool.loadData("/home/changuk/data/MovieLens/movielens.dat")
>>> from recommender import UserBased
>>> ubcf = UserBased()
>>> ubcf.loadData(data)
>>> import similarity
>>> simMeasure = similarity.cosine_intersection
>>> for user in data.keys():
...     recommendation = ubcf.Recommendation(user, simMeasure=simMeasure, nNeighbors=30)
```



上机练习

lidation

```
>>> import tool
>>> trainSet = tool.loadData("/home/changuk/data/MovieLens/u1.base")
>>> testSet = tool.loadData("/home/changuk/data/MovieLens/u1.test")
>>> from recommender import UserBased
>>> ubcf = UserBased()
>>> ubcf.loadData(trainSet)
>>> model = ubcf.buildModel(nNeighbors=30)
>>> import validation
>>> result = validation.evaluateRecommender(testSet, ubcf, model=model, topN=10)
>>> print(result)
'Precision': 0.050980392156862, 'Recall': 0.009698538130460, 'Hit-rate': 0.5098039215686}
```

